

Relatório de Implementação do Sistema de Autocomplete com Trie

Gustavo

1 Representação da Trie

A estrutura da Trie foi projetada com um alfabeto de tamanho 36. Essa decisão foi tomada para acomodar todos os caracteres alfanuméricos necessários para a chave de busca do sistema: as 26 letras do alfabeto inglês e os 10 dígitos numéricos (0 a 9).

Cada nó da Trie (**TrieNode**) possui um array estático de 36 ponteiros. O mapeamento do caractere para a sua respectiva posição (índice) no array ocorre da seguinte forma: as letras (a a z) ocupam os índices de 0 a 25. O cálculo do índice é feito pela subtração do valor na tabela ASCII: `indice = caractere - 'a'`. Já os números (0 a 9) ocupam os índices de 26 a 35, e o cálculo do deslocamento adiciona 26 à subtração: `indice = 26 + (caractere - '0')`.

A normalização de títulos e prefixos ocorre no método `toSearchKey`, responsável por gerar a chave de busca interna e garantir o comportamento *case-insensitive* e a ignorância de espaços. Quando um espaço é detectado, os caracteres à sua direita são deslocados uma posição para a esquerda (sobrepondo o espaço), e o último caractere é removido da string via `pop_back()`. As letras de 'A' a 'Z' são convertidas para minúsculas adicionando 32 ao seu valor ASCII, equiparando-as às minúsculas na busca.

2 Funcionamento dos Métodos

O método `insert` inicia calculando a chave de busca do título do jogo a ser inserido. A partir da raiz, ele itera sobre cada caractere da chave. Se o ponteiro correspondente ao índice do caractere for nulo, um novo **TrieNode** é instanciado. O ponteiro de navegação avança para o nó filho atualizado. Ao fim da iteração da chave, a *flag isEndOfTitle* do último nó é marcada como `true`, e o ponteiro para o objeto **Game** é armazenado nesse nó folha.

A busca por um título exato, através do método `contains`, converte a entrada para sua chave de busca e percorre a árvore seguindo os índices correspondentes a cada caractere. Se o ponteiro de navegação encontrar um filho nulo, a busca é interrompida e retorna `false`. Se conseguir navegar por todos os caracteres, o método verifica a propriedade `isEndOfTitle` do nó final, retornando `true` apenas se este caminho demarcar um título válido previamente inserido.

O fluxo do `autocomplete` é dividido em etapas. Primeiro, a árvore é percorrida utilizando a chave de busca do prefixo. Se o prefixo não existir na árvore, o método já retorna um vetor vazio. A partir do último nó do prefixo, uma travessia recursiva (*Depth-First Search*) é realizada em todos os caminhos possíveis, coletando os objetos **Game** sempre que `isEndOfTitle` for `true`. A lista coletada é submetida a um *Insertion*

Sort customizado, ordenando os jogos decrescentemente por popularidade e utilizando a ordem alfabética da chave de busca para desempate. Ao final, o vetor de resultados sofre um redimensionamento para garantir o retorno de, no máximo, k elementos.

3 Análise de Custo Computacional

Para a análise de custo, consideram-se os parâmetros: L como o tamanho do título, p como o tamanho do prefixo, s como o número de nós visitados na subárvore do prefixo, e r como o número de jogos encontrados.

A operação **insert** possui custo $O(L)$. Ela requer, no máximo, a criação de L nós correspondentes ao tamanho do título. A indexação nos arrays dos filhos é feita em tempo constante $O(1)$.

A operação **contains** também possui custo $O(L)$. Semelhante à inserção, a busca percorre no máximo os L caracteres do título. Caso exista incompatibilidade inicial, o tempo é ainda menor.

A operação **autocomplete** possui custo total estimado de $O(p + s + r^2)$. Descer a árvore até o fim do prefixo custa $O(p)$. Percorrer a subárvore via busca em profundidade visita s nós, gerando um custo de $O(s)$. Por fim, ordenar os r resultados encontrados com o algoritmo *Insertion Sort* custa $O(r^2)$ no pior caso.

4 Comparação e Uso de Memória

Uma busca de *autocomplete* ingênua utilizando um array consistiria em percorrer toda a base de jogos N e comparar os primeiros p caracteres do título com o prefixo buscado, resultando num custo assintótico em torno de $O(N \times p)$, seguido da ordenação. Em cenários onde o catálogo N é grande, essa varredura linear torna-se extremamente lenta. A Trie supera drasticamente essa abordagem ao isolar as buscas a um caminho específico, percorrendo apenas $O(p + s)$ e ignorando ramos da base de dados que não correspondem à chave solicitada.

No entanto, a performance em tempo da Trie exige um custo maior em memória. A solução ingênua em array gasta apenas o espaço contíguo para os objetos e seus dados. Já a Trie requer a alocação dinâmica de um nó para cada caractere inserido, além de manter os ponteiros para os filhos.

A redundância na criação de nós é mitigada pelo compartilhamento de prefixos. Títulos como “Hades”, “Halo” e “Half Life” compartilham a rota inicial de caracteres na árvore (“ha”). Quanto maior for o catálogo e mais os títulos compartilharem radicais, mais eficiente fica a estrutura da Trie na compactação dos caracteres armazenados, reduzindo o número total de nós instanciados.

Por outro lado, o tamanho estático do alfabeto causa desperdício de memória por nó. Como cada nó carrega um array fixo para `ALPHABET_SIZE = 36`, e muitos caminhos na árvore são singulares, ocorre a criação excessiva de ponteiros nulos. Assim, quanto maior o tamanho do alfabeto, maior será o consumo de memória da estrutura nas ramificações esparsas.